

Was ist die Speicherverwaltung?

Key Takeaways

- Speicherverwaltung ist eine zentrale Aufgabe des OS
- Speicherverwaltung kümmert sich um den Hauptspeicher
- Das OS muss wissen:
 - Was (welcher Prozess)
 - Wann
 - Wo gespeichert ist

Ein Computer braucht Speicher. Sollen Prozesse abgearbeitet werden, dann muss dieser irgendwo gespeichert sein. Die Daten, die ein Prozess manipulieren soll ebenso. Variablen, Heap, Stack, alles braucht Speicher.

Neben der Festplatte, welche im Kapitel des Dateisystems bereits besprochen wurde, gibt es den **Hauptspeicher / RAM / Arbeitsspeicher**. Dieser ist dafür da, Daten nur für die Dauer der Verwendung zu speichern, aber diese dafür schnell verfügbar zu haben.

Dieser Speicher muss von verwaltet werden, damit zu jeder Zeit bekannt ist, wie viel Speicher frei und belegt ist, wo der Speicher belegt ist, etc, vor allem da sich der Arbeitsspeicher schnell verändert. Diese Verwaltung fällt dem OS zu.

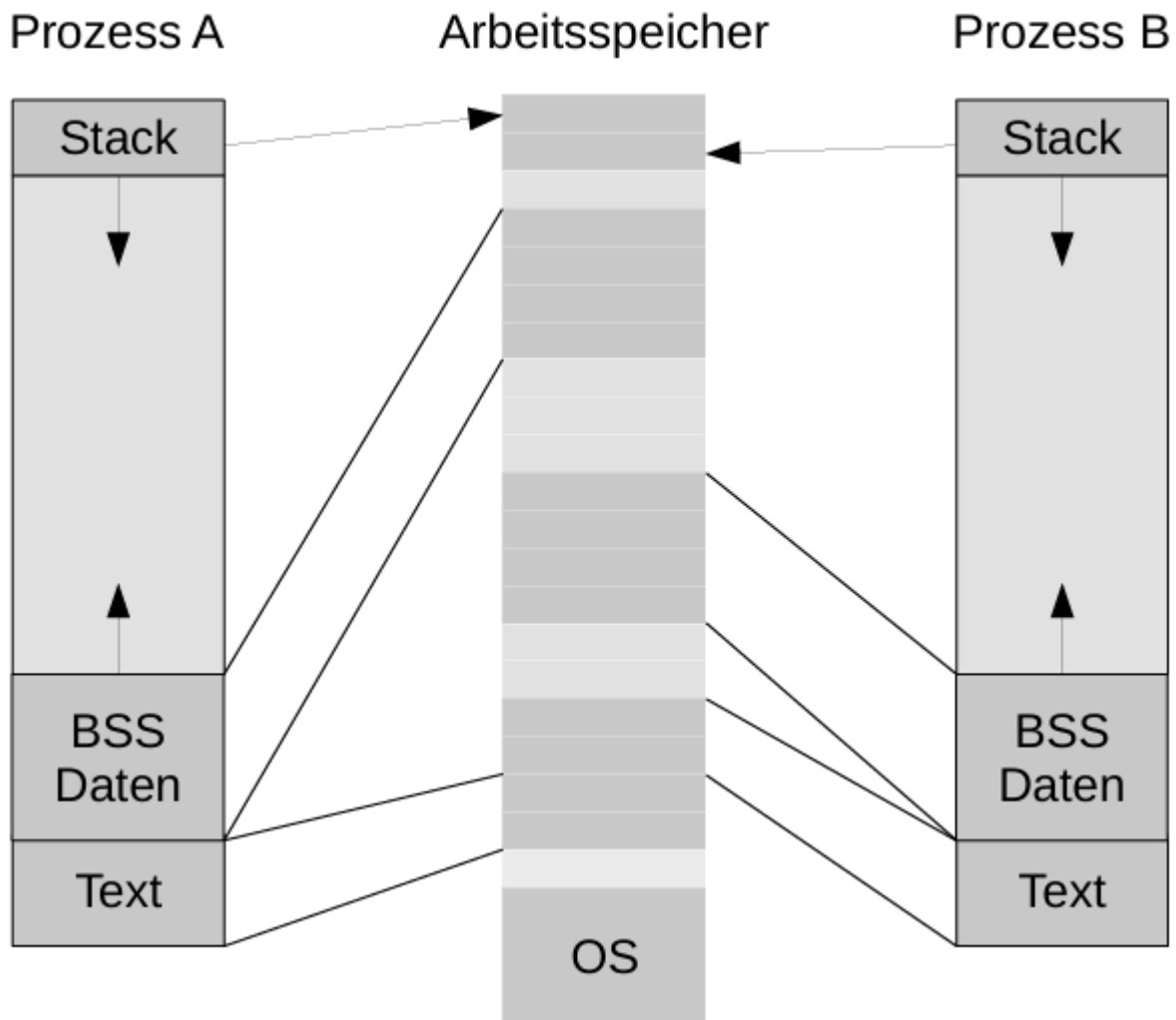
Die Hauptaufgaben der Speicherverwaltung sind:

- Zuteilung von Speicher zu Prozessen
- Allokieren / Freigeben von Speicher
- Auslagern (Paging) auf die Festplatte
- Optimale Nutzung des Speichers

Speicherlayout in Linux

In Linux werden Daten im Arbeitsspeicher an bestimmten Orten abgelegt.

- Ganz am Anfang werden die Stacks der einzelnen Prozesse abgelegt.
- Ganz unten liegen die Daten des OS
- Die restlichen Punkte ("Text" - Code), BSS (Heap, nicht initialisierte Variablen), und Daten werden im restlichen Arbeitsspeicher verteilt abgelegt, allerdings in sich wieder möglichst sequenziell nacheinander, damit die zusammenhängenden Daten nebeneinander liegen.



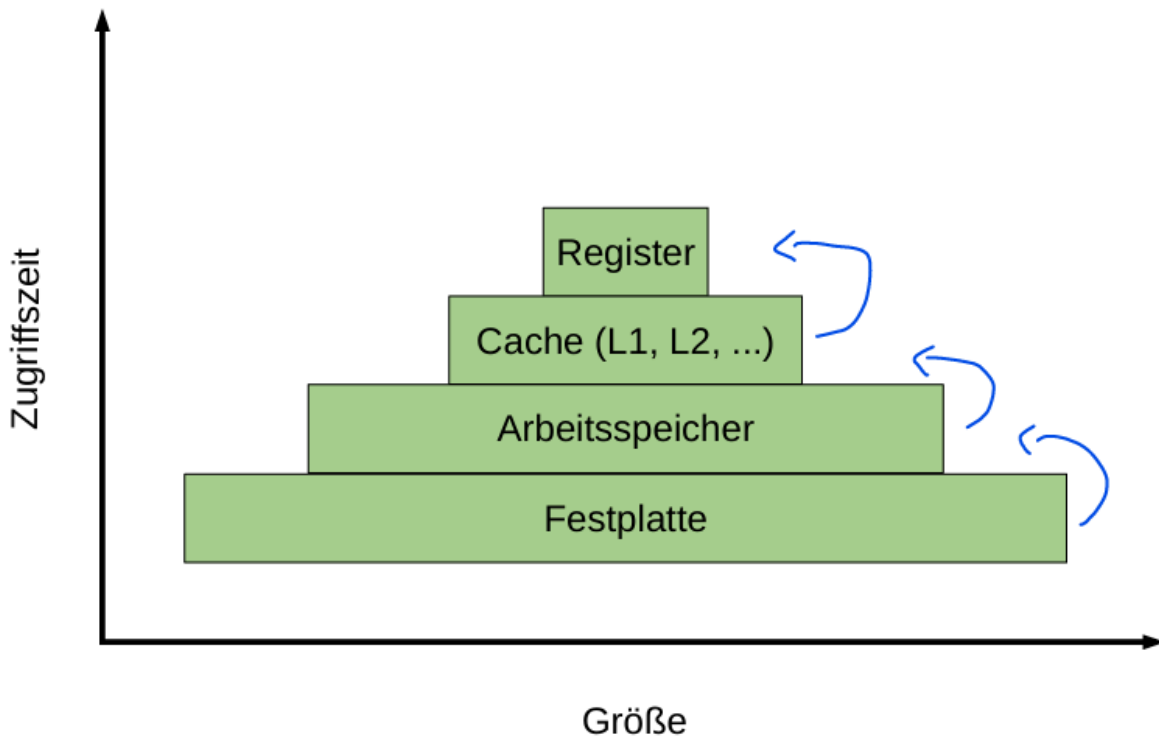
Die Speicherhierarchie

Key Takeaways

Die Speicherverwaltung arbeitet mit verschiedenen Speichern, die sich v.a in Speed und Größe unterscheiden.

- Register (Tiny, extremely fast)
- Cache (Very small, very fast)
- RAM (moderate size, moderate speed)
- Festplatte (gigantic, slow as shit)

Generell gibt es verschiedene Speicherarten, welche in einem PC vorliegen. Diese sind allesamt bereits bekannt, deshalb werden sie nur kurz angeschnitten:



- Register
 - Sehr kleine Speicher für ein Byte, extrem schnell von der ALU abfragbar. Dort landen die Informationen die auf Maschinencodeebene direkt für Rechenoperationen gebraucht wird
- Cache (L1, L2 ...)
 - Kleiner Speicher, welcher als Zwischenablage zwischen Arbeitsspeicher und CPU dient. Kleine Teile des Arbeitsspeichers werden dort abgelegt und können im Bedarfsfall ca. 100 Mal schneller abgefragt werden.
- Arbeitsspeicher
 - Der Arbeitsspeicher ist ein temporärer, moderat großer Speicher, wo alle Daten abgelegt werden, welche für Prozesse nötig sind. Der Zugriff ist immer noch schnell, aber der Speicher ist deutlich begrenzter. Dafür schädigt der Zugriff den Speicher nicht so stark.
- Festplatte
 - Großer, im Vergleich sehr langsamer Speicher für alle Daten, welche erhalten werden sollen. Kann zur Not auch als Erweiterung des Arbeitsspeicher verwendet werden, was allerdings schlecht für die Festplatte selbst ist.

Abstraktion

 **Der Arbeitsspeicher ist eine Aneinanderreihung von Speicheradressen**

- Prozesse wissen von alleine nicht, wo sie liegen
- Abstraktion erlaubt die Speicherung mehrerer Prozesse im Speicher

- Abstraktion erleichtert die Speicherzuweisung für Prozesse

Abstraktion ist ein wichtiges Konzept der Speicherverwaltung. Man kann den Arbeitsspeicher wie ein großes Array an Adressen betrachten, wobei jede Adresse die gleiche Menge Daten speichern kann. Da man in diesem Speicher arbeiten muss (herumspringen, Dinge umspeichern, löschen etc.) muss irgendwie bekannt sein, wo was im Speicher liegt, und vor allem wo ich (der Prozess) selber liege. Auf der Hardware selbst ist das allerdings extrem schwierig zu machen. Daher gibt es verschiedene Möglichkeiten den Speicherzugriff bzw. das Adressenmanagement zu abstrahieren und gut implementierbar zu machen.

Keine Abstraktion

Key Takeaways

Historischer Ausgangspunkt der Speicherverwaltung, alles wird 1 zu 1 physisch abgebildet

- Prozesse können sich gegenseitig beeinflussen
- Nur ein Prozess am gesamten Speicher
- Sehr ineffizient und gefährlich
- Schon lange nicht mehr wirklich genutzt

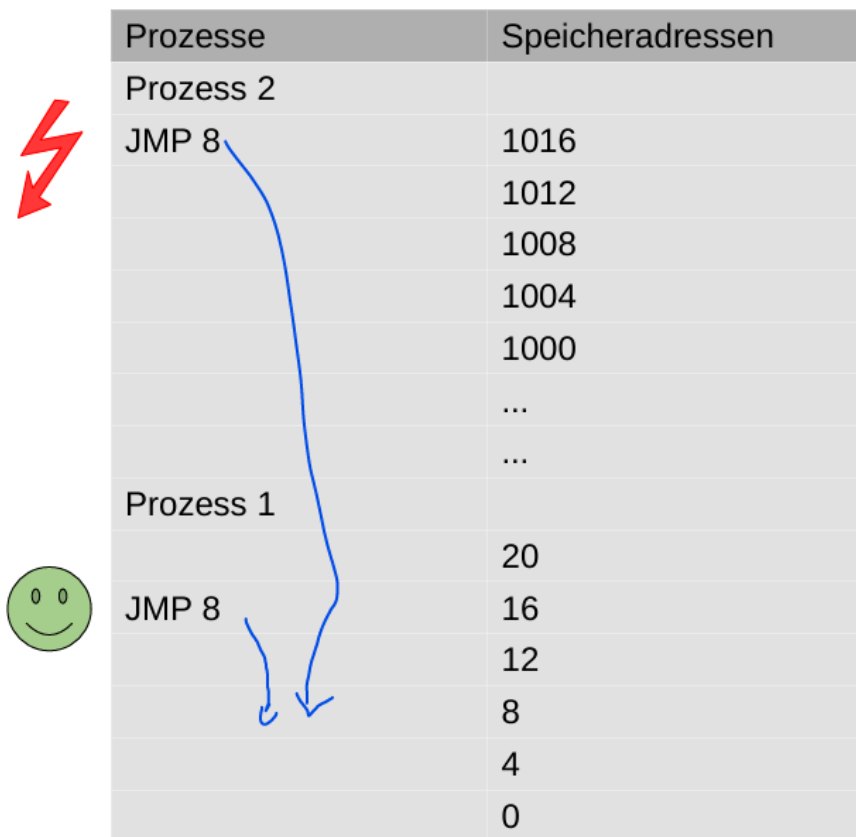
Früher gab es keine Abstraktion, jede Änderung, welche ein Prozess im Speicher gemacht hat, wurde dort ausgeführt wo es im Prozess geschrieben stand. Wollte der Prozess an die Stelle 8 springen, ist er real am Speicher an die Stelle 8 gesprungen. Unabhängig davon ob oder was dort eigentlich gespeichert lag.

Das hat zur Folge, dass real nur ein Programm gleichzeitig im Arbeitsspeicher liegen kann, da alle Sprünge absolut sind und ein weiterer Prozess im Speicher kein Wissen darüber hat, wo er im Speicher liegt.

Das ist natürlich ein gigantisches Problem, immerhin wollen wir mehr als nur ein Programm gleichzeitig im Speicher haben können. Daher musste eine Abstraktionsebene her, welche dies erlaubt.

Beispiel für keine Abstraktion

Im folgenden Beispiel:



Prozesse	Speicheradressen
Prozess 2	
JMP 8	1016
	1012
	1008
	1004
	1000
	...
	...
Prozess 1	
	20
JMP 8	16
	12
	8
	4
	0

Prozess rechnet absolut (also von 0 weg)
Deshalb sind die Sprünge immer absolut -> Kann in andere Programme hineinspringen weil er nicht weiß, wo er startet.

zeigt sich das oben beschriebene Problem:

Der Prozess 1 wird zuerst in den Speicher geladen. Er belegt die Adressen 0 bis 20.

+ An der Stelle 16 wird ein JMP 8 Befehl aufgerufen. Instruction Pointer geht zu Adresse 8. Alles Gut!

Ein Prozess 2 wird in den Speicher geladen. Da 0-20 bereits belegt sind, landet er an einer anderen Stelle im Speicher (hier 1000 - 1016)

+ An der Stelle 1016 liegt eine JMP8 Anweisung. Der Instruction Pointer geht zu Adresse 8.

+ PROBLEM: Hier liegt der Prozess 1, nicht Prozess 2 wie angenommen!

Die CPU führt also plötzlich Befehle von Prozess 1 aus. Dies führt im besten Fall zu ungewolltem Verhalten, im schlimmsten Fall zu irreparablen Schäden an den Daten!

Da diese Art der Speicherverwaltung, wie im Beispiel gezeigt, gigantische Probleme hat und große Einschränkungen mit sich bringt, wurden schnell andere, abstraktere Ideen entwickelt.

Adressräume

Adressräume ermöglichen mehrere Prozesse in einem Hauptspeicher und verhindern Zugriff zwischen Prozessen

- Dazu werden zwei Grenzen gespeichert
 - Base Register - Startadresse des Prozesses
 - Limit Register - Endadresse des Prozesses
- Alle Speicherzugriffe werden relativ zum Prozessstart verschoben und geprüft ob sie im Adressraum liegen
- Nachteil: Bei jedem Speicherzugriff muss ich rechnen

Die Abstraktion durch Adressräume versucht das Problem der absoluten Sprünge zu beheben und ermöglicht es, mehrere Prozesse gleichzeitig im Arbeitsspeicher geladen zu haben.

Die Idee funktioniert wie folgt:

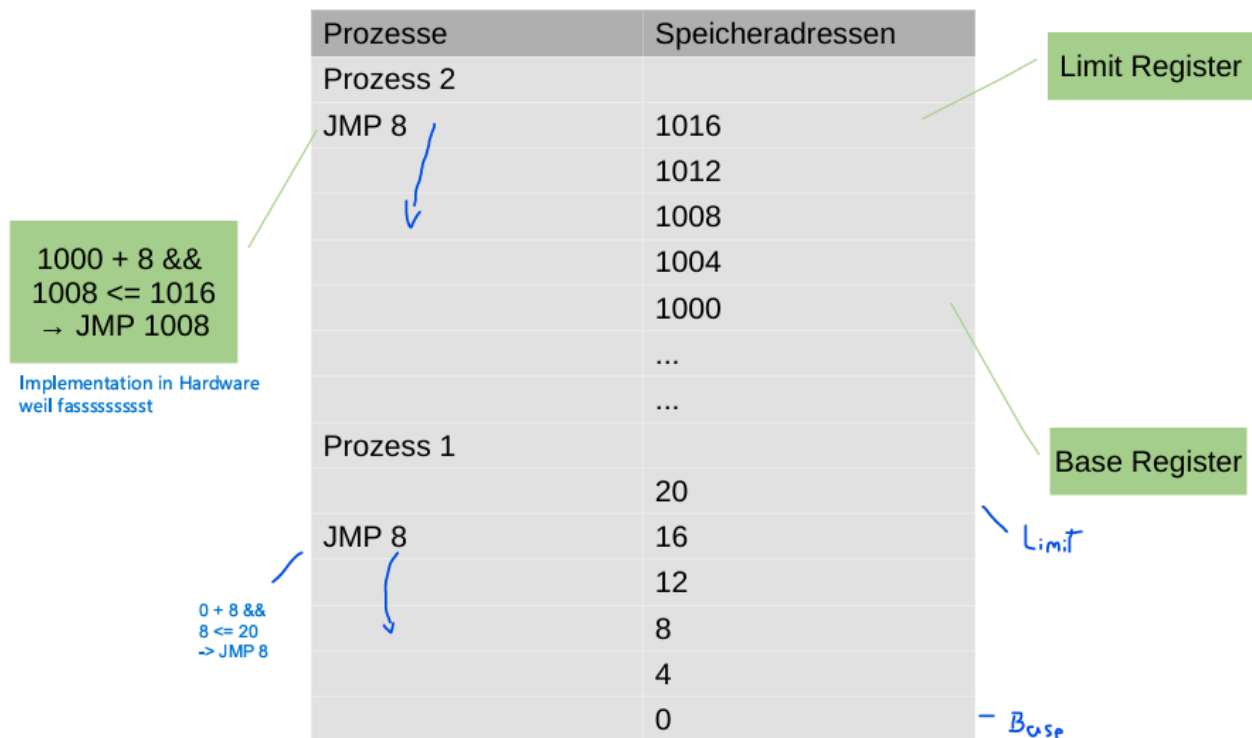
- Wird ein Programm in den Speicher geladen, merkt sich das OS zwei Adressen:
 - Base Register: Die kleinste Speicheradresse / die Startadresse des Prozesses im Speicher
 - Limit Register: Die größte Speicheradresse / die Endadresse des Prozesses im Speicher
- Alle Adressen zwischen Base und Limit (inkl. Base und Limit) sind der Adressraum des Prozesses.
- Will der Prozess nun eine Speicheradresse ansprechen (wie oben zB. durch JMP 8), macht das OS folgende Dinge:
 - Es addiert das Base Register zu der geforderten Adresse (verschiebt die Adresse quasi relativ zum Programm im Speicher)
 - Es vergleicht, ob die neue Adresse größer ist als die Limit-Adresse. (Prüfung, ob Adresse im Adressraum liegt)
- Ist der Check erfolgreich, gibt das OS dem Befehl statt und er wird ausgeführt
- Ist der Check nicht erfolgreich, verweigert das OS die Ausführung des Befehls

Durch dieses System wird sichergestellt, dass sich Programme immer nur in ihrem eigenen Adressraum bewegen können und diesen nicht verlassen. Da sich Programme im Speicher nicht überschneiden sind somit ungewollte Manipulationen ausgeschlossen!

Der Nachteil ist, dass mit diesem System bei jedem Speicherzugriff eine zusätzliche Addition und ein Vergleich ausgeführt werden müssen, was den Zugriffsaufwand **mind. verdreifacht!**

Durch Adressräume können nun mehrere Programme zugleich im Speicher arbeiten!

Dynamic Relocation - Beispiel



Swapping

Key Takeaways

Wenn der limitierte Arbeitsspeicher voll wird, kann ich Prozesse auf die Festplatte auslagern

- Prozesse werden auf die Festplatte gespeichert und bei Bedarf wieder geladen
- Erlaubt mehr Prozesse als eigentlich möglich
- Ist aber sehr langsam
- Es ist schwer zu entscheiden, welchen Prozess man entladen kann

Nun haben wir das Problem der Speicherzugriffe durch Prozesse gelöst (vorerst). Nun könnten wir erfreut alle unsere Prozesse in den Speicher laden. Irgendwann würden wir allerdings an eine weitere Grenze stoßen:

Arbeitsspeicher ist begrenzt.

Gerade bei großen Prozessen (oder ineffizienten keks wie Chrome), wird der Arbeitsspeicher schnell voll. Um Prozesse korrekt verwenden zu können müssen wir sie trotzdem irgendwie speichern. Dafür gibt es Möglichkeiten:

- Wir könnten einfach sobald ein neuer Prozess geladen wird der mehr Platz bräuchte als da ist, einen anderen Prozess beenden. Das wäre jedoch kein sonderlich schönes Feeling für den User.

Will man das Schließen von Prozessen vermeiden, kann man auf eine andere Technik zurückgreifen: **Swapping**

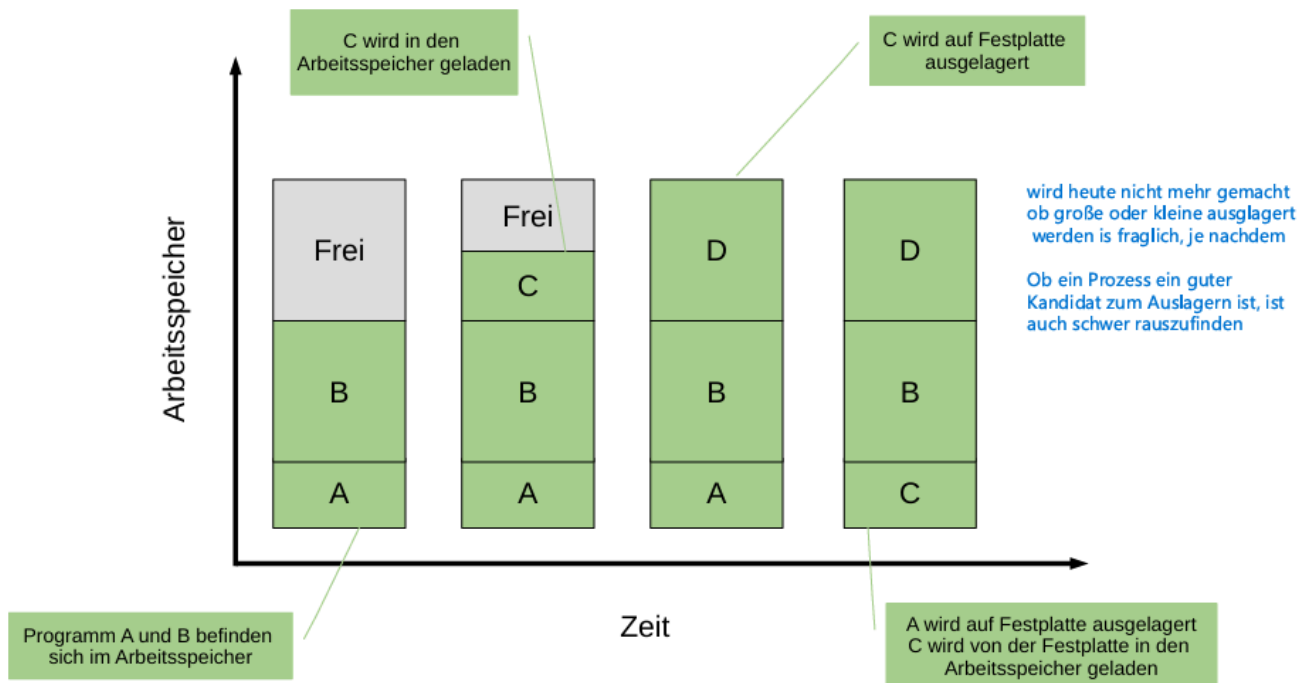
Swapping bezeichnet eine Methode, Prozesse zeitweise auf die Festplatte zu speichern, und sie später wieder in den Arbeitsspeicher zu laden, wenn sie gebraucht werden. Das ganze funktioniert so:

- Soll ein neuer Prozess geöffnet werden, schaut das OS nach, welche Prozesse gerade nicht aktiv laufen.
- Davon wählt es einen aus.
 - Die Auswahl zu treffen ist schwer. Man könnte mehrere kleine oder einen großen Prozess nehmen.
 - Das Hauptproblem ist, dass das OS nicht wissen kann, welchen Prozess der Nutzer bald wieder braucht.
 - Das ist ein Problem, weil das OS gerne die Prozesse auslagern würde die lange ungebraucht bleiben, da das wieder hereinladen natürlich sehr lange dauert.
- Hat es eine Wahl getroffen, werden die Daten auf die Festplatte gelegt und der Arbeitsspeicher mit den neuen Daten des Prozesses überschrieben.
- Will der Nutzer den ausgelagerten Prozess wieder verwenden, muss das OS wieder durch Swapping für diesen Platz machen, vorausgesetzt es ist nicht genug offener Speicher vorhanden.

Die Hauptprobleme des Swapping sind, dass das OS nicht wissen kann wie sich der User verhält, und dass der Festplattenzugriff einfach sehr, sehr langsam ist. Deshalb wird Swapping in dieser Form heutzutage auch nicht mehr so eingesetzt. (zumindest in dieser rudimentären Form)

Swapping - Beispiel

Swapping - Beispiel



Speicherdarstellung 1: Block A und B liegen im Speicher, es ist noch etwas frei

Speicherdarstellung 2: Block C wird in den Speicher geladen (kein Swapping)

Speicherdarstellung 3: Block D soll in den Speicher geladen werden

- + OS muss entscheiden, welchen Prozess es auslagert
- + OS entscheidet sich für C, da er klein ist und genug Platz macht
- + OS lädt alle Daten von C auf die Festplatte, und überschreibt im RAM die Daten von C mit denen von D.

Speicherdarstellung 4: User will Prozess C wieder verwenden:

- + OS muss entscheiden, welchen Prozess es auslagert
- + OS entscheidet sich für A, da er klein ist und genug Platz macht
- + OS lädt alle Daten von A auf die Festplatte, und überschreibt im RAM die Daten von A mit denen von C.

Der User wird ab dem Laden von C (Darstellung 4) Verzögerungen gemerkt haben.

:C

Das Problem mit der Speicher(m)allokierung/Swapping with Holes

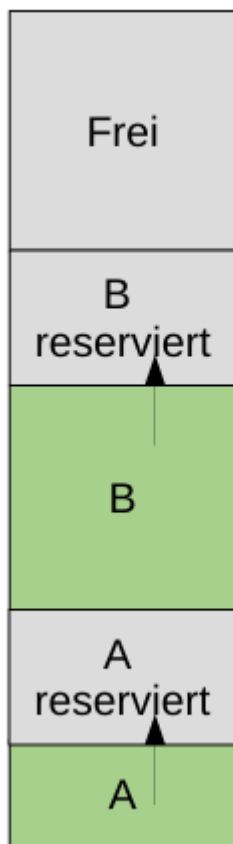
Key Takeaways

Real sind Prozesse dynamisch, sie allokalieren Speicher

- Damit ein Prozess wachsen kann braucht er reservierten Speicherplatz
- Daher muss immer eine Lücke zwischen Prozessen bleiben
- Schlecht, da weniger Prozesse pro Speicher
- Überschreitet der Prozess seinen reservierten Platz, muss er umgespeichert werden.

Bisher sind wir davon ausgegangen, dass Prozesse im Speicher eine fixe Größe haben. In der Realität ist das jedoch nicht der Fall, da Prozesse durch das Laden von Daten, durch das Initialisieren von Variablen und Datenstrukturen und das Löschen dieser zur Laufzeit ihre Größe ändern.

Daher muss einem Prozess mehr Platz eingeräumt werden, als er beim Hineinladen in den Speicher potentiell brauchen würde. Da Prozesse am Hauptspeicher möglichst nebeneinander liegen sollen, um so wenig wie möglich springen zu müssen (HDDs und Adressraum), müssen wir also einen Teil des Speichers **reservieren**, da es viel langsamer wäre für jeden Wachstumsvorgang den ganzen Prozess neu zu speichern.



Dieser Fakt hat den Nachteil, dass **weniger Prozesse gleichzeitig im RAM liegen können, da jeder Prozess einen größeren Adressraum hat als zuvor**

Weiters kann es passieren, dass der **reservierte Platz zu klein ist**.

- Ist dies der Fall, wird versucht den Prozess an eine andere Speicherstelle mit genügend Platz zu schreiben (langsam, aber machbar da schneller RAM Zugriff)
- Gibt es das nicht, muss mit Swapping wieder Platz geschaffen werden (Verzögerung)
Hier wird nur der tatsächlich beschriebene Speicher des Prozesses gewapped, und nicht leerer, reservierter Speicher auch.
- Geht auch das aus irgendwelchen Gründen nicht, kommt es zu einem Programmabsturz (OutOfMemoryException)

Memory Management (Keeping Track of Memory)

Hinweis

Dieser Teil der Speicherverwaltung wurde lt. Scheibenpflug noch nie bei der Klausur gefragt

Key Takeaways

Analog zum Dateisystem muss das OS wissen, welcher Speicher belegt ist

- Bitmaps -> konstante Größe, schnell
- Verkettete Listen -> variable Größen, werden je nach Bedarf anders durchsucht (meist First Fit)

Wir haben jetzt das Problem mit dem Speichern mehrerer, dynamischer Prozesse (halbwegs) gelöst. Oft haben wir dabei davon gesprochen, dass das OS Prozesse an "passend große, leere Stellen im Speicher" schreibt. Dafür muss das OS aber erstmal **wissen, welche Stellen im Speicher belegt sind und welche nicht!**

Hierfür verwendet es folgende Techniken (welche uns aus der Dateiverwaltung bereits sehr bekannt vorkommen, da es basically die gleichen sind)

Bitmaps

Eine Bitmap (also ein Array aus 0en und 1en) wird für den gesamten Hauptspeicher erstellt. Hierbei wird nicht für jedes Bit am Speicher ein Bit auf der Bitmap gesetzt sondern für jede **Allocation Unit (welche die Größe einer Speicheradresse ist)**

Je kleiner diese Units sind, desto größer muss dementsprechend die Bitmap sein, da es insgesamt mehr Adressen im Speicher gibt.

Bsp:

Angenommen wir haben einen Arbeitsspeicher von 32GB und eine Allocation Unit von 4Byte.

$4 \text{ Byte} = 4 * 8 \text{ Bit} = 32 \text{ Bit}$

$32 \text{ GB} = 32 * 1024 \text{ MB} = 32 * 1024 * 1024 \text{ KB} = 32 * 1024 * 1024 * 1024 \text{ Byte} =$
 $32 * 1024 * 1024 * 1024 * 8 \text{ Bit} = 274\,877\,906\,944 \text{ Bit}$

$274\,877\,906\,944 / 32 = 8\,589\,934\,592 \text{ Adressen}$

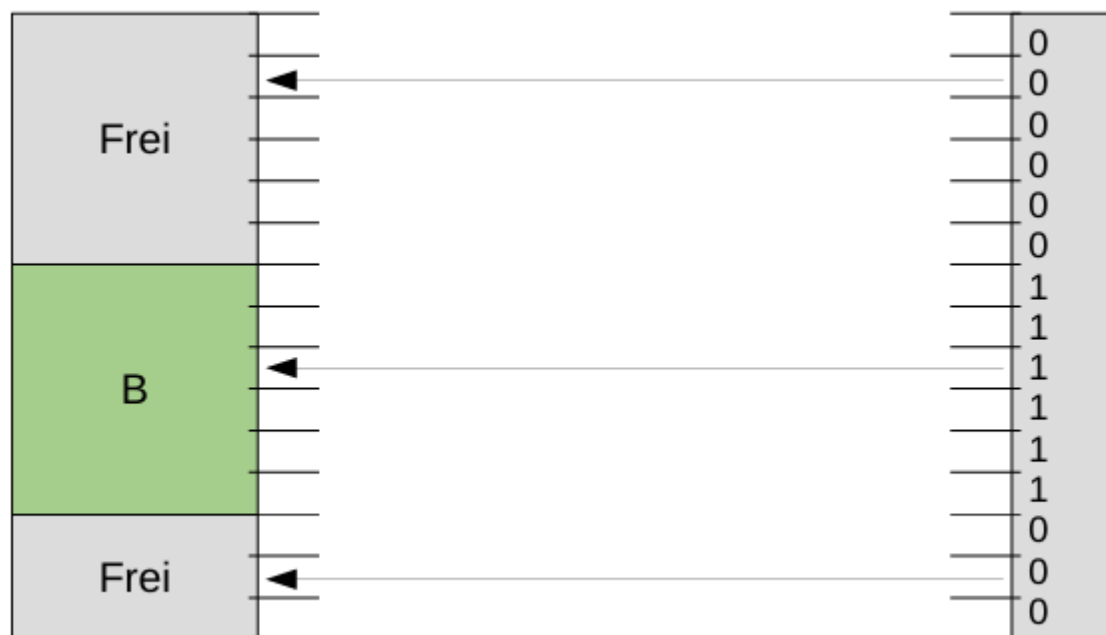
Eine Bitmap hat ein Bit pro Adresse:

Bitmap Größe = $8\,589\,934\,592 \text{ Bit} = 1\,073\,741\,824 \text{ Byte} = 1\,048\,576 \text{ KB} = 1\,024 \text{ MB}$
= 1GB

Wäre die Allocation Unit 2 Byte, wäre die Bitmap 2GB groß

Wäre die Allocation Unit 8 Byte, wäre die Bitmap 512MB groß

Für freien Speicher wird eine 0, für belegten Speicher eine 1 gespeichert.

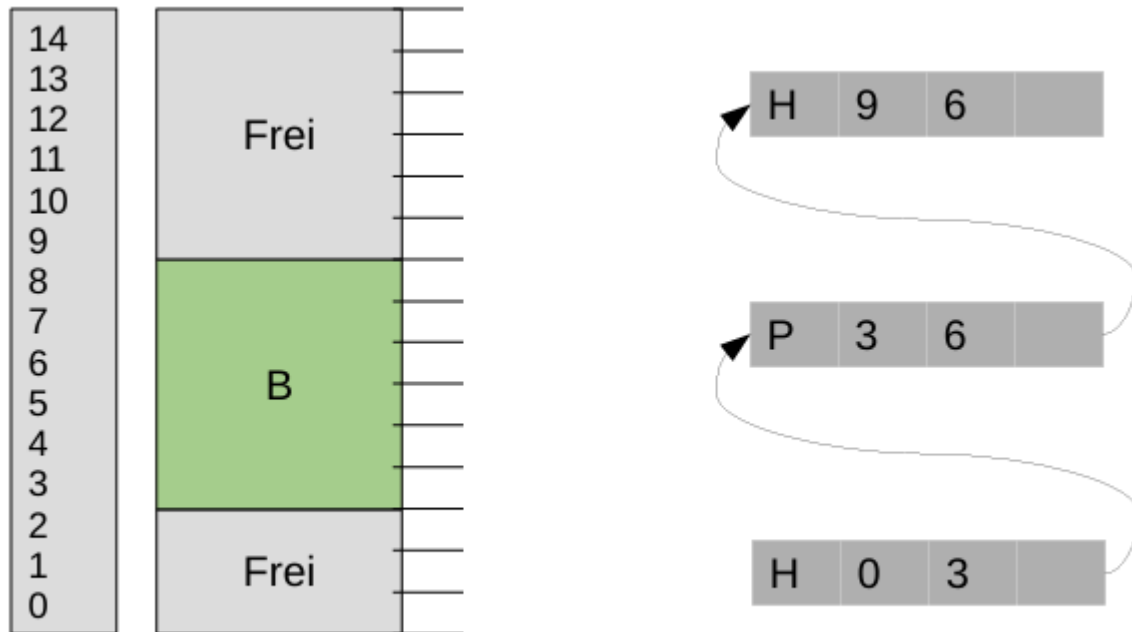


Verkettete Listen

Wie auch in der Dateiverwaltung kann die Position von freien Datenblöcken auch durch Listen gespeichert werden. Hierbei werden in einer verketteten Liste 2 Knotentypen gespeichert.

- Process -> Speicher, welcher von einem Prozess belegt ist
- Hole -> Speicher, welcher frei ist.

Dabei wird in beiden die Beginn und die Endadresse gespeichert. Die Länge der Liste hängt von der Prozess und Lochanzahl ab und nimmt mit Fragmentierung zu. (Viele kleine Löcher = viele kleine Knoten und Prozesse)



Hierbei muss entschieden werden, welcher freie Platz (Hole Knoten) für einen neuen Prozess verwendet werden soll. Dafür gibt es mehrere Ansätze:

- First Fit -> Ich speichere in das erste Loch mit genug Platz
- Best Fit -> Ich suche im gesamten Speicher das Loch, welches am ehesten zur Prozessgröße passt.

Best Fit tendiert dazu, sehr kleine Speicherlöcher zu produzieren (da Prozesse selten genau in ein Loch passen) und braucht länger, da er erst die ganze Liste durchsuchen muss.

First Fit tendiert zu größeren Speicherlücken, da es nicht auf die Größe des Speichers achtet solange er größer als die prozessgröße ist.

Allerdings sind große Speicherlücken später vllt noch nutzbar, obendrein ist First Fit schneller, dementsprechend wird eher first fit verwendet.

Virtual Memory

Key-Takeaways

Virtual Memory ist eine Abstraktion der physischen Speicherung, welche es erlaubt mehr Prozesse zu speichern als eigentlich am Hauptspeicher gehen sollte.

- Durch virtuelle Adressräume mehr Adressen als physisch
- Durch Pages und Paging kann ich Programmteile einzeln auslagern
- Der MMU berechnet aus virtuellen Adressen physische Adressen

- Der TLB und Page Table behalten die Referenzen zwischen Pages und Frames

Das ganze ist kompliziert, bitte lest die langversion xd

Virtual Memory verwendet die oben erklärten Techniken (Swapping, Adressräume) und führt eine weitere Abstraktionsebene ein, welche das Zugreifen auf physischen Speicher schneller und effizienter macht. Der Hauptvorteil des Virtual Memory ist das **Paging**, welches es ermöglicht einen Prozess im RAM abzuarbeiten, ohne dass der komplette Prozess im RAM liegen muss.

Die Bestandteile des Virtual Memory werden im folgenden genauer erläutert:

Pages und Paging

Die Kernidee des Virtual memory ist folgende:

Wenn ein Programm durchlaufen wird, muss nicht zu jeder Zeit das gesamte Programm zur Verfügung stehen.

Man möchte also einen Prozess in mehrere Teile aufteilen, damit man manche von ihnen bei Bedarf auslagern kann, um Speicherplatz zu sparen, bzw mehr Prozesse auf gleichem Platz zu verwenden.

Diese Prozessteile nennen sich **Pages**. Sie haben alle eine feste Größe und beinhalten einen Teil des Prozesses. Wichtig ist, **Pages werden auf dem virtuellen Adressraum gebildet**. Im virtuellen Adressraum wird also Page nach Page abgelegt. Korrespondierend dazu gibt es einen **Page Frame**, welcher gespeichert hat wo sich die Information der Page auf dem physischen Speicher befindet.

Der Vorgang, eine Page auf den Hauptspeicher auszulagern bzw. sie später wieder ins RAM zu laden nennt sich **Paging**.

Die Idee funktioniert so:

- Prozess fordert Daten von einer Adresse
- Es wird diese Adresse im virtuellen Raum gesucht, und geschaut in welcher Page diese liegt
- Dann wird geschaut, ob diese Page einen passenden Page Frame im RAM hat
 - wenn ja, wird die Information geladen
 - wenn nein, wurde der passende Frame zuvor ausgelagert (swapped)
 - der passende Frame wird wieder hereingeladen
 - die Operation wird ausgeführt

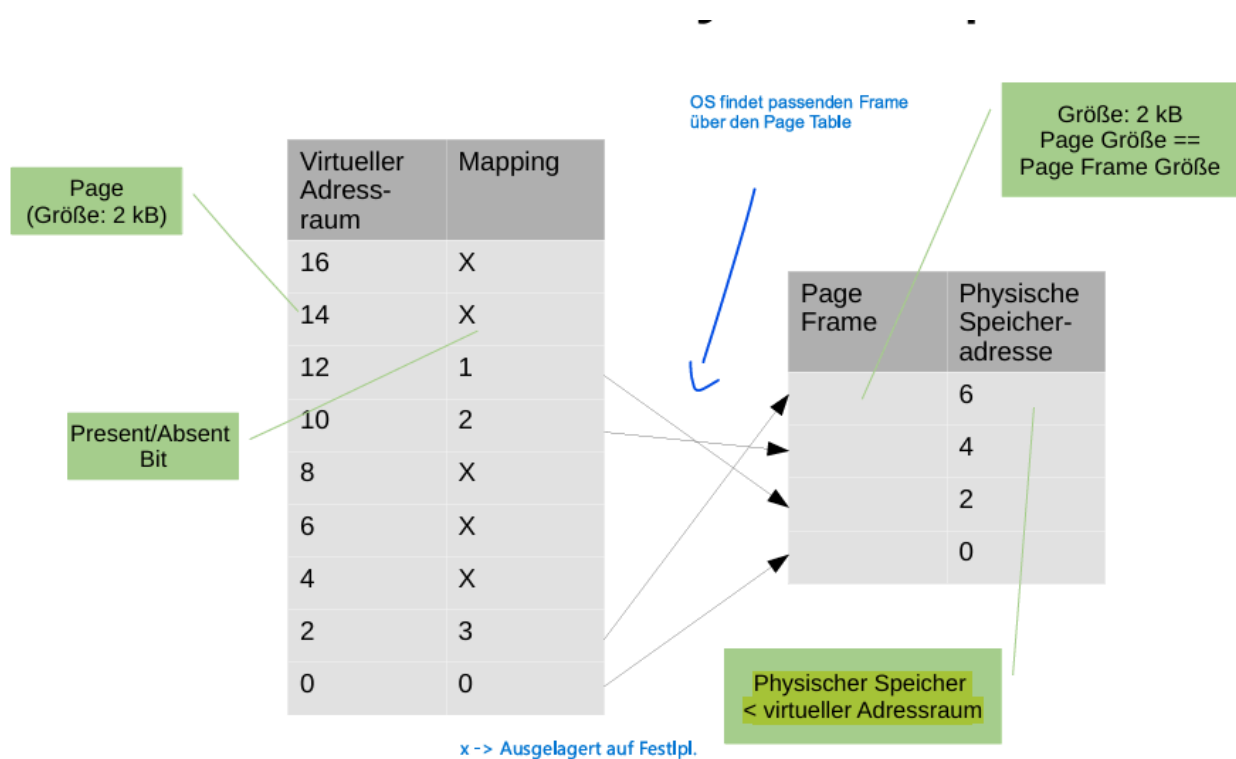
Virtuelle und physische Adressräume

Eine Hauptidee des Virtual Memory ist es, einen neuen, virtuellen Adressraum einzufügen, auf welchen die Prozesse zugreifen. Die Befehle der Prozesse, welche eigentlich auf physische Adressen gehen, werden vom Compiler auf den virtuellen Adressraum angepasst.

Das bedeutet:

- Ein Prozess schickt will zu einer Speicheradresse springen
- Er schickt den Befehl mit Adresse an die CPU
- Der Compiler ändert die physische Adresse auf die passende Adresse im virtuellen Adressraum um.
- Anstelle sofort auf den physischen Speicher zu gehen, wird im virtuellen Speicher die Adresse angesteuert
- Es wird ermittelt, in welcher Page die Adresse liegt
- Von hier aus alles wie unter "Pages / Paging"

Der virtuelle Adressraum ist **größer** als der physische Adressraum. Das muss er auch sein, immerhin will man mehr Pages referenzieren können als auf dem physischen Speicher Platz haben. Wären sie gleich groß, wäre die Technik nutzlos.



Der virtuelle Adressraum bekommt eine Page Nummer zugewiesen (Hier als Mapping bez.)

Der Page-Frame hat ebenfalls die Page Nummer gespeichert, sowie den dazu passenden physischen Adressraum. Ist die Page Nummer X (absent), dann weiß man, dass die Page derzeit nicht auf dem Hauptspeicher liegt. Page Frames können logischerweise nur Pages referenzieren die sich auch auf dem physischen Speicher befinden

Ist also die passende Page nicht am RAM, muss diese nachgeladen werden. Dafür kommt die MMU ins Spiel. (Kommt etwas später.)

Page Table

Der PageTable ist eine Hilfsstruktur für die MMU. Es handelt sich um eine Tabelle, welche Informationen zu den Pages und den dazugehörigen Page Frames speichert. Ohne diese Tabelle könnte das MMU die Page Frames nicht finden. Die wichtigsten Informationen sind das Mapping von Page auf Page Frame, sowie der Offset innerhalb der Frame-Adressierung. **Pro Prozess gibt es einen Page Table** (sofern keine Multilevel Page Tables verwendet werden, später mehr)

- Mapping: Zwei Einträge, einmal die virtuelle Adresse der Page, und einmal die physische Adresse des Page Frames
- Offset: Eine Bitmenge, welche angibt, welche Adresse innerhalb der Page gesucht wird.

Der Page Table selbst liegt am Arbeitsspeicher, was den Zugriff durch die CPU etwas langsam macht. Um dieses Problem zu minimieren, wird der Translation Lookaside Buffer (TLB) verwendet.

Page Table Einträge

Ein Page Table Eintrag besteht aus folgenden Informationen

- Page Number (Die virtuelle Adresse der Page, wird als Index in der Page Tabelle verwendet)
- Frame Page Number (Die Adresse des Page Frames) (leer wenn Page ausgelagert)
- Offset (Bitfolge, welche es ermöglichen im einzelnen Frame Adressen anzusprechen)
- **Present / Absent Bit:** Bit welches angibt, ob sich der Frame derzeit innerhalb des Arbeitsspeichers befindet oder nicht.
- Protection bit: Gibt an, ob es Lese- oder Schreibzugriff gibt
- Modified bit: Gibt an, ob der Page Inhalt verändert wurde und somit vom OS auf die Festplatte gespeichert werden muss
- Referenced bit: Gibt an, ob ein Frame in letzter Zeit angesprochen wurde, was ihn zu einem schlechten Kandidaten für einen Swap macht (OS geht davon aus, dass wenn ein Frame vor kurzem gebraucht wurde, dass er bald wieder gebraucht wird.)

Page Number	Frame Number	Present / Absent Bit	Protection Bit	Modified Bit	Referenced Bit
0000	1010	1	...	...	...
0001	1111	1			
0010	1011	1			
0011	0001	1			
0100	null/empty	0			
0101	null/empty	0			
0110	...	...			
0111					
1000					
1001					
1010					
1011					
1100					
1101					
1110					
1111					

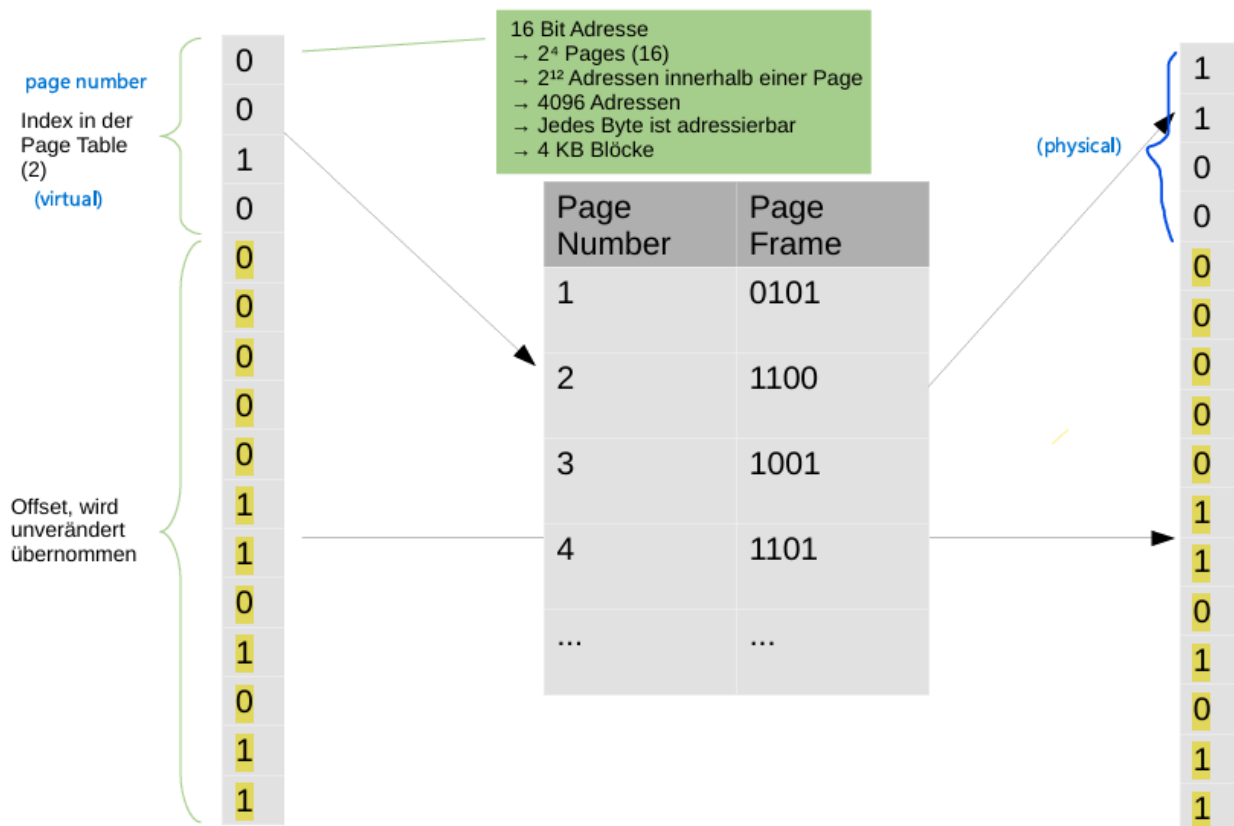
Translation Lookaside Buffer

Der TLB ist eine kleine Tabelle in der MMU. Er hat die Aufgabe, das Durchsuchen des Page Tables schneller zu machen. Oft greifen Programme nur auf wenige Pages oft zu. Der TLB dient quasi als Cache indem er bereits verwendete Page Table Einträge zwischenspeichert. Die MMU geht immer zuerst den TLB durch, und schaut ob sie die gewünschte Page findet (TLB Hit). Erst wenn sie im TLB nicht fündig wird, sucht sie im gesamten Page Table. Findet die MMU den Eintrag dann, wird er gelesen und im TLB abgelegt, damit er bei einem erneuten Aufruf schneller gefunden werden kann.

Das Ziel, genau wie bei normalen Caching, ist es Zugriffszeit zu sparen.

MMU - Memory Management Unit

Die MMU, Memory Management Unit, ist ein Teil der CPU und hat den Job, die virtuellen Adressen, welche es aus dem Page Tables bekommt, auf die physischen Adressen, welche es aus den Page Frames entnimmt, zu mappen. Dazu verwendet es die TLB und den Page Table. Findet es den passenden Page Frame in einer der beiden Strukturen, baut es die virtuelle Adresse in eine physische Adresse um, indem es die virtuelle Adresse vom Offset trennt, und stattdessen die physische Adresse anfügt. Diese Information kann nun an den Arbeitsspeicher weitergegeben werden.

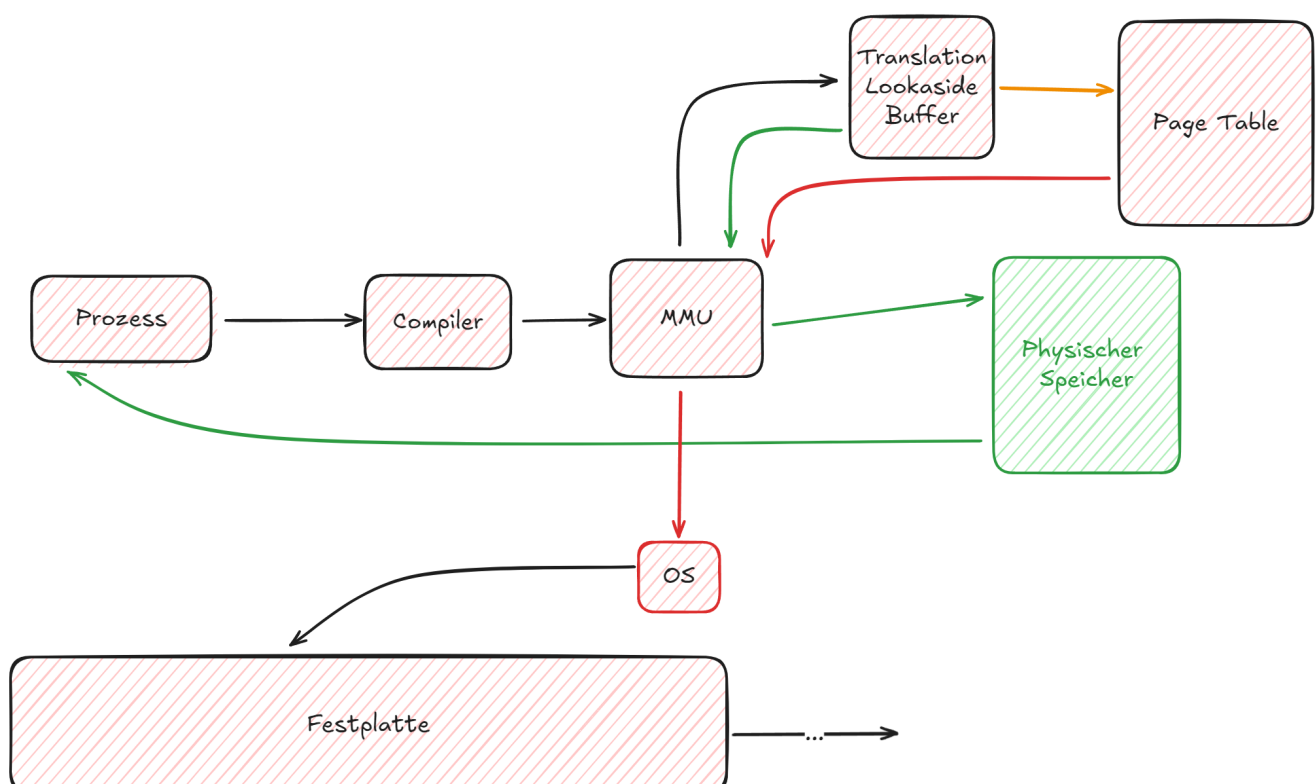


Die obersten 4 Bit (links) ist die virtuelle Adresse

Die unteren 12 Bits sind der Offset

Findet die MMU in den Page Table oder TLB den richtigen Eintrag, wird der passende Page Frame genommen und anstatt der virtuellen Adresse eingesetzt.

Beispiel - Speicherzugriff eines Prozesses mit Virtual Memory



1. Der Prozess schickt eine Anfrage mit einer Adresse los
2. Der Compiler macht daraus die passende virtuelle Adresse
3. Die MMU bekommt die Adresse
4. Die MMU schaut im TLB nach, ob sich der passende Eintrag darin befindet
grün: TLB Hit, MMU bekommt die passende Page Number
gelb: TLB Miss, MMU sucht im Page Table (schaut ob present / absent bit = 1)
5. grün Nach TLB Hit: MMU baut die Adresse um und leitet sie an Speichertreiber weiter
6. grün Nach TLB Miss und Page Table Hit (PA-Bit = 1): MMU baut die Adresse um und schickt sie an Speichertreiber weiter, speichert Adresse im TLB
7. rot: Das PA-Bit war auf 0, MMU muss OS informieren (Page Fault)
8. OS reagiert, indem es den passenden Frame im Hauptspeicher sucht
9. Frame wird in den RAM gespeichert (wenn nicht genug platz werden andere frames auf die Festplatte ausgelagert)
10. Das OS aktualisiert den Page table (setzt PA-Bit auf 1 und speichert Frame Number)
11. OS startet informiert den MMU, der Befehl nochmal ausführt.

Performance

Key Takeaways

Da virtual memory möglichst effizient sein muss, gibt es Techniken die das System noch verbessern.

- Multilevel Page Tables sparen viel Speicher, da keine leeren virtuellen Adressen mehr gespeichert werden müssen
- Translation Lookaside Buffer speichert bereits verwendete Page Adressen ab und dient der MMU als Buffer

Die Performance der oben erklärten Umwandlung ist entscheidend, da sie bei jedem Speicherzugriff gebraucht wird. Um die (sehr großen) Page Tables nicht immer durchsuchen zu müssen, gibt es den Translation Lookaside Buffer (TLB). Es gibt allerdings noch andere Verbesserungen, welche zu schnellerer Abarbeitung dieser Umwandlung beitragen:

Multilevel Page Tables

Page Tables können extrem groß werden:

zB: Virtuelle Adresse: 32 Bit

Page Größe = 4KB

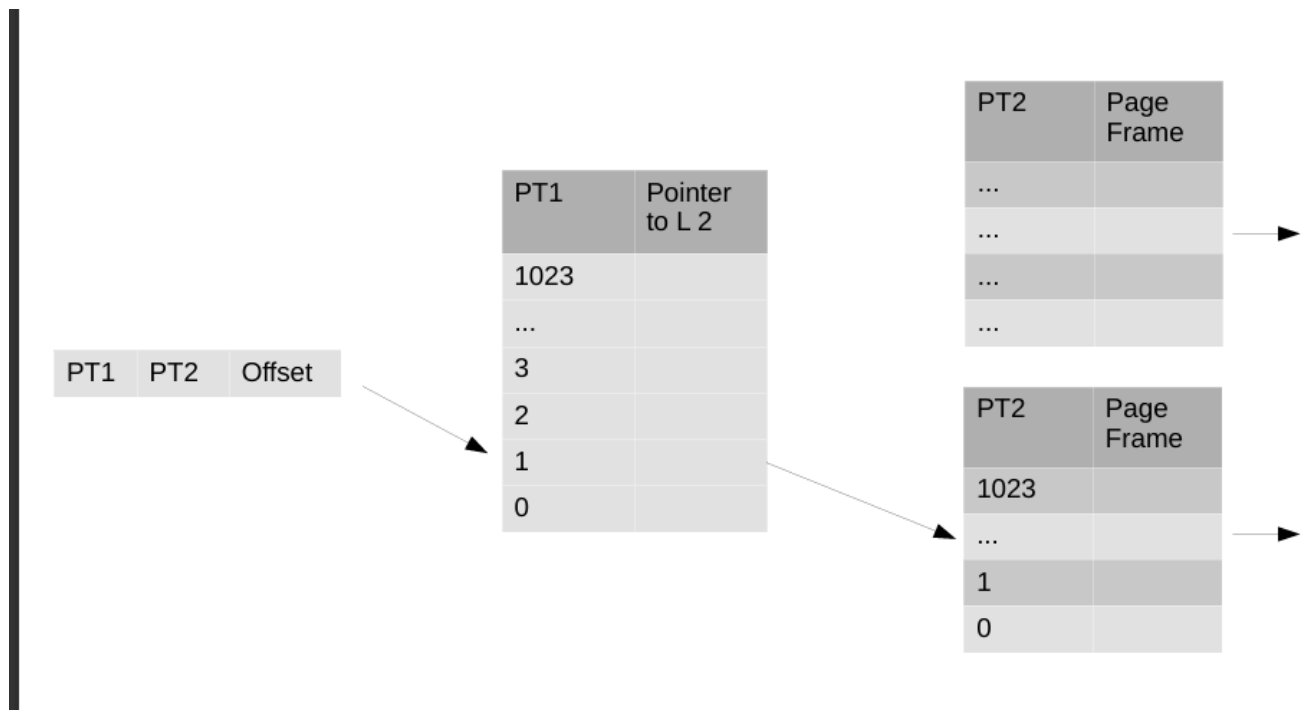
$4KB = 2^{12} \text{Bit} \rightarrow 12 \text{ Bits der Adresse sind der Offset}$

$32 - 12 \text{ sind } 20 \text{ Bits für die Page Number}$

Da jede Page Number einen Eintrag braucht, wären dass 2^{20} Einträge $\sim 1 \text{ Mio}$
Bei 64 Bit systemen wäre das der Wahnsinn.

Da jede Number einen Eintrag braucht, wären die Tabellen auf modernen Systemen unnutzbar groß. Daher hat man sich entschieden, anstatt eines großen Tables ein Multilevel System einzuführen, welches eine Directory Tabelle hat, die dann auf kleinere Page Tables verweist.

Die Idee ist, nur die tatsächlich verwendeten Pages abzulegen, anstatt alle möglichen. Der Aufbau ähnelt sehr stark den Multilevel I-Nodes:



Inverted Page Tables

Dreht die Idee eines Page tables um, heißt anstatt die virtuelle number als Index zu nehmen wird die physische verwendet. Das macht die Größe der Tabelle konstant, da der physische Speicher gleich groß bleibt. Ist aber arsch langsam und auch die TLB hilft da ned viel

Ersetzungsstrategien

 Key-Takeaways

Da das OS oft Pages auf dem Hauptspeicher ersetzen muss, gibt es verschiedene Ansätze dafür

- NRU: Schaut sich die referenced und modified bits an ersetzt jene, die am wenigsten referenzed / modified sind

Wie oben beschrieben, wenn die MMU keine passende Frame Adresse bauen kann, muss das OS die Page in den Hauptspeicher laden. Ist dieser voll, muss sie andere Frames herausnehmen und dann wieder auf die Festplatte speichern.

Dieser Ersetzungsvorgang muss wissen, welche Frames er ersetzen kann, möglichst ohne Gefahr zu laufen sie bald wieder hineinladen zu müssen (hey, wir sind wieder bei dem Problem von weiter oben "Swapping" angekommen)

Um diese Entscheidung effizient zu treffen, gibt es mehrere Methoden:

NRU - Not Recently Used

Hier kommen die Modified und Referenced Einträge des Page Tables ins Spiel. Das OS schaut sich auf der Suche nach passenden Swap-Kandidaten die Tabelle an und sortiert die Pages in folgende Klassen:

- Klasse 0: Nicht referenziert, nicht modifiziert
- Klasse 1: Nicht referenziert, modifiziert
- Klasse 2: referenziert, nicht modifiziert
- Klasse 3: referenziert, modifiziert

Die Idee ist, wie oben bei Swapping auch, dass eine vor kurzem referenzierte Page wahrscheinlicher wieder gebraucht wird als eine nicht referenzierte, gleiches für modifiziert.

NRU nimmt also von Klasse 0 die Pages der niedrigsten Klassen als Swap Kandidat und speichert diese auf die Festplatte, um Platz für die neuen Pages zu machen.

Die referenced Bits werden nach einer gewissen Zeit (Timer Interrupt) resettet, heißt einmal referenzierte Pages sind nicht für immer referenziert.

Die modified Bits werden erst zurückgesetzt, wenn die Änderungen vom OS auf die Festplatte gespeichert wurden.

Der Rest des Foliensatzes (Ab Folie 33 - FIFO - Second Chance) ist nicht mehr prüfungsrelevant